



PART 1 · INTRO

CONSTRUCTION · AGENTIC AI · DATA INSIGHT

Creating Robust Agentic AI Systems to Increase Data Insight Accessibility

Also known as: *Building Agentic Solutions for Construction*

ACCESSIBLE INSIGHT · ANALYTICS ADOPTION · PRODUCTION-GRADE AGENTS

The practitioner's journey

HPC foundation (~10 years): shipping software products, taking them to market, and learning **hyperscale** reliability—customers across **manufacturing, life sciences, oil & gas, aerospace, defense, and automotive**. That shaped how I think about scalable systems.

Cloud shift: as HPC workloads moved cloudward, the work became **architecting cloud-native applications** and **native integrations**—the same discipline, different substrate.

FlyPaper (CTO): **BIM coordination, superintendent dailies, and pull planning** software grounded in the **Last Planner System**—mobile and web products built to meet crews in the field.

Procore acquisition → Barton Malow: helped stand up **data and automation**, built the **engineering team**, ran early “**what is AI / how do we use it?**” research, shipped internal **orchestration** work, and moved with the industry toward **MCP-style** tool contracts.

Today: building **agentic AI** solutions for **AEC strategy professionals**—with a special focus on **competitive intelligence tools** that empower **BD and strategic decision-makers**, as well as on enhancing **preconstruction workflows** for greater efficiency and insight.

Orchestration — the central nervous system

In 2026 the battleground is **who controls context and workflow**—not which foundation model “wins.”

Strategy: choose **off-the-shelf** when the problem already fits the vendor’s graph and speed beats differentiation. Choose **hands-on** when workflow, data, economics, or **deep internal business knowledge** must be yours—or you share the same packaged intelligence as everyone else.

LLM = engine · orchestration = transmission

OFF-THE-SHELF

- **Databricks Agent Bricks + Unity Catalog** — agents, tools, and lakehouse data in **one** governed loop (identity, MCP catalog, observability)
- **Procore Helix** (Assist, Agent Builder, **Datagrid** / connectors)—excellent **in-platform** loop; weak where edge lives in **local LOB** spreadsheets / systems outside the graph
- **Glean, Bedrock Agents** — “managed factory”: you land data; they run the thinking loop (COIs, standard retrieval)
- **Autodesk Assistant, Claude Cework, Zapier**—class hubs — **utility** productivity; **vendor tax** / metering for speed—fine when the vendor boundary is acceptable

HANDS-ON (POWER MOVE)

- **Workflow you encode** — treat logic as a **state machine**, not a wall of prompt. Example: **submittal** — **spec** → **extract** → **compare** → if **deviation > threshold** → **human**. Stack: **LangGraph** / **LangChain**, **AWS Agent Core**, **Temporal** + **n8n/Windmill** for durable glue
- **Typed + multi-agent** — **Pydantic AI** (ERP-safe I/O), **CrewAI** (roles and handoffs)
- **Integration boundary** — **custom MCP** (Python/Node) for wholesale APS / Procore APIs vs throttled vendor MCP; connect **internal + third-party** systems and **lake/warehouse** views so stakeholders can **ask data** without living in SQL—full story on the MCP slide
- **Product surface** — **Vercel AI SDK** + bespoke UI (e.g. superintendent: **Check safety** · **Summarize daily** · **Order materials**)—tools, not a thread.

Beyond the chatbot

Role-appropriate surfaces — not “chat bad,” but “chat is one option”

Conversational analytics: when the surface *is* chat, **context and specificity** (what project, what time range, which source of truth) **directly shape** output quality—brief users once; don’t assume “the AI knows.”

Field vs. trailer: voice-forward chat can fit **hands-busy field** roles; a **project engineer** on an iPad or laptop in the trailer often needs **structured** views, approvals, and diff—not a thread.

When chat is the wrong default: if the job is “produce an artifact” (report, package, leveled bid), **ship the artifact UI**—a dashboard or wizard that **thinks**, instead of making people type prompts.

Labor study — product vs. chat

Not only a chatbot that “explains BLS.” **Ship** an interactive **labor study** —BLS (and peers) fused with your assumptions, **AI synthesis**, exportable narrative—so users manipulate the **report**, not the prompt box.

Submittals — distribution lists vs. experience

Classic **email + distribution lists** work until volume and revision churn win. A **unified web/mobile** submittal surface can **abstract AI** behind routing, redlines, and audit trails—**outcome-first**.

Vendor agents vs. rolling your own

MCP is one way to wire tools — the business question is **who owns the boundary**

Vendor path: Procore / Autodesk / others ship **agents, connectors, and “opinionated” flows**—fast to adopt, often **metered**, roadmap is **theirs**. You pay for the abstraction; it may not optimize for **your** margin.

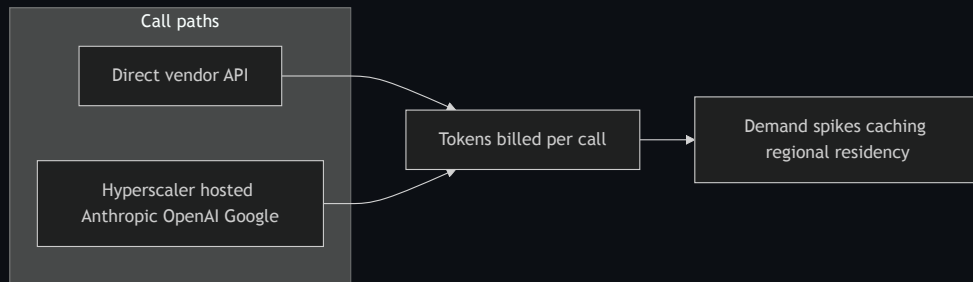
Build path: Custom **integrations** (including **MCP servers**) across **internal systems and third-party platforms**—APS, ERP read models, **lake / warehouse** views—**your** auth, caching, pagination, and **policy**. Sometimes you can **beat** packaged assistants on **latency and unit economics**—sometimes not; **prototype both**.

Decision lens: **technical capability** and **strategic fit**; integration **surface area** (how many systems?), **compliance**, **rate limits**, and whether the vendor’s “happy path” matches your **WBS / CM** reality.

Own the boundary you cannot afford to rent: lineage, approvals, and where **context** is assembled.

Inference, tokens, and scale

Where you run matters as much as which model



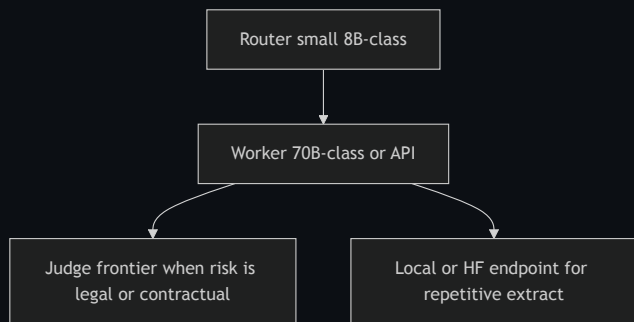
Prototype vs. production: direct Anthropic / OpenAI APIs for benching; Bedrock / Azure OpenAI / Vertex for procurement, guardrails, and **data residency**.

Economics move: Anthropic models run on **AWS**—list vs. Bedrock pricing **crosses over** over time as deals change. Re-benchmark **quarterly**; the “cheaper path” is not a constant.

Tokens = capacity planning: batch size, concurrency, **prompt caching** for stable corpora (spec books), and **back-pressure** when vendors throttle.

Model selection — right engine, right hill

Models are stochastic—**reliable systems** come from **router / worker / judge**, schemas, and **eval loops**, not from pretending the LLM is a calculator.

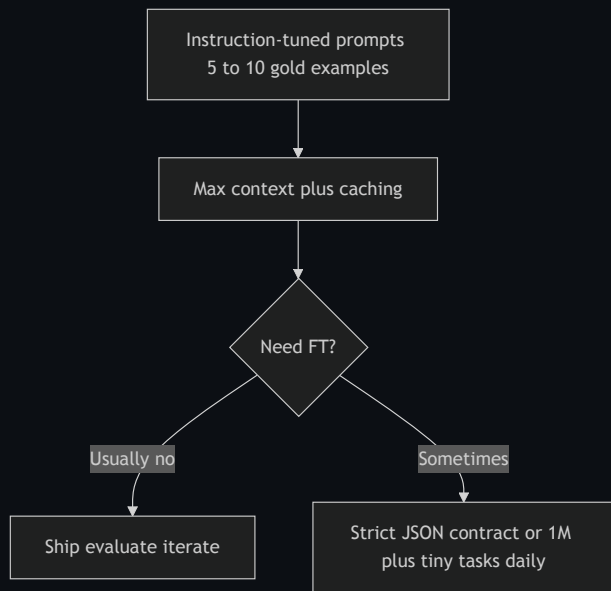


Use the **smallest** reliable model per step — **tokens are margin**, and latency is trust.

Hugging Face & self-host: great for **exploration**, **air-gapped** constraints, and **high-volume** narrow tasks —know your **ops** cost, not just API list price.

Anti-pattern: “Claude for everything” is **using a haul truck to catch a mouse**—fun demos, expensive production.

The fine-tuning trap



Before fine-tuning, exhaust: ICL + **full context** (manuals, standards packs) with **caching** where available.

Fine-tune when: you need a **hard format contract** or **massive** repetitive micro-tasks where **latency/cost** dominates.

Scenario cards

Group 1 — Silent submittal auditor

PROBLEM Reviewers miss alternates buried in long vendor PDFs.

TASK Agentic flow that flags mismatches against the **master spec** — **without** a chat-first UI.

Group 2 — Bid leveling engine

PROBLEM Twenty trade bids with different exclusions and scopes.

TASK Use **MCP** to pull APS model or property data and normalize bids into a standard **WBS** view.

Group 3 — Historical knowledge estimator

PROBLEM Junior staff lack narrative memory of **why** a past job bled margin.

TASK Tool that reads **actuals** from ERP via MCP to sanity-check a new estimate.

MORE WORKSHOP IDEAS — SWAP IN FOR A FUTURE SESSION OR SPARE GROUP

Alternate — Invoice vs. contract

PROBLEM Draws, pay apps, and vendor invoices drift from **contract** / **SOV** language—finance catches it late.

Critique rubric

After groups present — use this to score designs in ~2 minutes each. Favor ideas with a **clear pilot metric** (time saved, defect rate, adoption) so spend stays justified.

1. **Architecture:** router/worker/judge — or one monolithic call?
2. **UX:** chatbot by default, or a tailored surface?
3. **Data:** what is the **source of truth** and who can access it?
4. **Monday plan:** what is the **30-day pilot** (scope, metrics, owner)?
5. **Outcome:** what **measurable** result unlocks the next funding or scale gate?

Final thought

Your job is not to be an AI **user**. It is to be a **practitioner**: build bridges, own context, and protect your company's intelligence.

Data insight accessibility · AEC · Own the graph

<https://www.linkedin.com/in/ispyhumanfly/> or email me at dan@thoughtpivot.com.